Python Code

```python
###Q3: allreduce###
###please implement ring_allreduce method, using  pytorch's dist method is not allowed###

from torch._utils import _flatten_dense_tensors, _unflatten_dense_tensors
import torch
import torch.distributed as dist

def reduce_scatter(chunks, tmp, world, rank, left, right):

    # your code here: follow slides instruction: do counter-clockwise iteration
    for i in range(world - 1):
        send_idx = (rank - i) % world
        recv_idx = (rank - i - 1) % world

        s = dist.isend(chunks[send_idx], dst=right)
        r = dist.irecv(tmp, src=left)
        s.wait()
        r.wait()

        chunks[recv_idx].add_(tmp)
    cur = (rank - world + 1) % world
    return cur

def all_gather(chunks, tmp, current, world, rank, left, right):
    # your code here: follow slides instruction: do counter-clockwise iteration
    for i in range(world - 1):
        s = dist.isend(chunks[current], dst=right)
        r = dist.irecv(tmp, src=left)
        s.wait()
        r.wait()

        nxt = (current - 1) % world
        chunks[nxt].copy_(tmp)
        current = nxt
    return

def ring_allreduce_(tensor: torch.Tensor, world_size = None, rankid = None):
    """In-place ring all-reduce (SUM, optional average) using isend/irecv."""
    world = world_size
    if world == 1: return tensor
    rank = rankid
    left, right = (rank - 1) % world, (rank + 1) % world

    ##following steps try to fill blank to the tensor so that final tensor can be divided to 3 chunks evenly
    flat = tensor.contiguous().view(-1)
    n = flat.numel()
    chunk = (n + world - 1) // world
    pad = chunk * world - n
```

```python
    # your code here: we cannot divide flat into 3 pieces evenly as the
    # flat lengh may not be able to divided exactly by 3....
    #So, fill zeros at the end of flat to generate padded_flat
    padded_flat = None # modify this line and fill correct value into padded_flat
    if pad > 0:
        padded_flat = torch.cat([flat, flat.new_zeros(pad)], dim=0)
    else:
        padded_flat = flat

    chunks = [padded_flat[i*chunk:(i+1)*chunk] for i in range(world)]
    tmp = torch.zeros_like(chunks[0])
    # your code here: call reduce_scatter and all_gather
    current = reduce_scatter(chunks, tmp, world, rank, left, right)
    all_gather(chunks, tmp, current, world, rank, left, right)

    #we provide the reduce_scatter and all_gather func prototype for you
    # You may adjust the function signature (input structure) of `reduce_scatter` and `all_gather` if needed.
    flat = padded_flat
    # stitch & unpad
    flat /= world
    tensor.view(-1).copy_(flat[:n])
    return
```